

Practical7

September 24, 2026

```
[1]: import pandas as pd
from sklearn.datasets import load_breast_cancer

#Load dataset
data = load_breast_cancer(as_frame=True)

#Convert to dataframe
df = data.frame

#Display initial information
print("First five rows:
```

```
Cell In[1], line 11
    print("First five rows:
      ^
SyntaxError: unterminated string literal (detected at line 11)
```

```
[2]: import pandas as pd
from sklearn.datasets import load_breast_cancer

#Load dataset
data = load_breast_cancer(as_frame=True)

#Convert to dataframe
df = data.frame

#Display initial information
print("First five rows:")
print(df.head())

print("\nData Shape:", df.shape)

print("\nMissing values:", df.isnull().sum().sum())

print("\nClass distribution:")
```

```
print(df["target"].value_counts())
```

First five rows:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
0	0.27760	0.3001	0.14710	0.2419	
1	0.07864	0.0869	0.07017	0.1812	
2	0.15990	0.1974	0.12790	0.2069	
3	0.28390	0.2414	0.10520	0.2597	
4	0.13280	0.1980	0.10430	0.1809	

	mean fractal dimension	...	worst texture	worst perimeter	worst area	\
0	0.07871	...	17.33	184.60	2019.0	
1	0.05667	...	23.41	158.80	1956.0	
2	0.05999	...	25.53	152.50	1709.0	
3	0.09744	...	26.50	98.87	567.7	
4	0.05883	...	16.67	152.20	1575.0	

	worst smoothness	worst compactness	worst concavity	worst concave points	\
0	0.1622	0.6656	0.7119	0.2654	
1	0.1238	0.1866	0.2416	0.1860	
2	0.1444	0.4245	0.4504	0.2430	
3	0.2098	0.8663	0.6869	0.2575	
4	0.1374	0.2050	0.4000	0.1625	

	worst symmetry	worst fractal dimension	target
0	0.4601	0.11890	0
1	0.2750	0.08902	0
2	0.3613	0.08758	0
3	0.6638	0.17300	0
4	0.2364	0.07678	0

[5 rows x 31 columns]

Data Shpe: (569, 31)

Missing values: 0

Class distribution:

target	
1	357
0	212

Name: count, dtype: int64

```
[3]: print("Column names:")
      print(df.columns)

      print("\nData types:")
      print(df.dtypes)

      print("\nDataset information:")
      df.info()
```

Column names:

```
Index(['mean radius', 'mean texture', 'mean perimeter', 'mean area',
       'mean smoothness', 'mean compactness', 'mean concavity',
       'mean concave points', 'mean symmetry', 'mean fractal dimension',
       'radius error', 'texture error', 'perimeter error', 'area error',
       'smoothness error', 'compactness error', 'concavity error',
       'concave points error', 'symmetry error', 'fractal dimension error',
       'worst radius', 'worst texture', 'worst perimeter', 'worst area',
       'worst smoothness', 'worst compactness', 'worst concavity',
       'worst concave points', 'worst symmetry', 'worst fractal dimension',
       'target'],
      dtype='object')
```

Data types:

mean radius	float64
mean texture	float64
mean perimeter	float64
mean area	float64
mean smoothness	float64
mean compactness	float64
mean concavity	float64
mean concave points	float64
mean symmetry	float64
mean fractal dimension	float64
radius error	float64
texture error	float64
perimeter error	float64
area error	float64
smoothness error	float64
compactness error	float64
concavity error	float64
concave points error	float64
symmetry error	float64
fractal dimension error	float64
worst radius	float64
worst texture	float64
worst perimeter	float64

```

worst area                float64
worst smoothness          float64
worst compactness         float64
worst concavity           float64
worst concave points      float64
worst symmetry            float64
worst fractal dimension   float64
target                    int64
dtype: object

```

Dataset information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 569 entries, 0 to 568

Data columns (total 31 columns):

#	Column	Non-Null Count	Dtype
0	mean radius	569 non-null	float64
1	mean texture	569 non-null	float64
2	mean perimeter	569 non-null	float64
3	mean area	569 non-null	float64
4	mean smoothness	569 non-null	float64
5	mean compactness	569 non-null	float64
6	mean concavity	569 non-null	float64
7	mean concave points	569 non-null	float64
8	mean symmetry	569 non-null	float64
9	mean fractal dimension	569 non-null	float64
10	radius error	569 non-null	float64
11	texture error	569 non-null	float64
12	perimeter error	569 non-null	float64
13	area error	569 non-null	float64
14	smoothness error	569 non-null	float64
15	compactness error	569 non-null	float64
16	concavity error	569 non-null	float64
17	concave points error	569 non-null	float64
18	symmetry error	569 non-null	float64
19	fractal dimension error	569 non-null	float64
20	worst radius	569 non-null	float64
21	worst texture	569 non-null	float64
22	worst perimeter	569 non-null	float64
23	worst area	569 non-null	float64
24	worst smoothness	569 non-null	float64
25	worst compactness	569 non-null	float64
26	worst concavity	569 non-null	float64
27	worst concave points	569 non-null	float64
28	worst symmetry	569 non-null	float64
29	worst fractal dimension	569 non-null	float64
30	target	569 non-null	int64

dtypes: float64(30), int64(1)

memory usage: 137.9 KB

```
[4]: X = df.drop(columns=["target"])
     y = df["target"]

     print("X shape:", X.shape)
     print("y shape:", y.shape)
```

X shape: (569, 30)
y shape: (569,)

```
[5]: from sklearn.model_selection import train_test_split

     X_train, X_test, y_train, y_test = train_test_split(
         X, y,
         test_size=0.20,
         random_state=42,
         stratify=y
     )

     print("X_train shape:", X_train.shape)
     print("X_test shape:", X_test.shape)
     print("y_train shape:", y_train.shape)
     print("y_test shape:", y_test.shape)
```

X_train shape: (455, 30)
X_test shape: (114, 30)
y_train shape: (455,)
y_test shape: (114,)

```
[6]: from sklearn.preprocessing import StandardScaler

     scaler = StandardScaler()

     X_train_scaled = scaler.fit_transform(X_train)
     X_test_scaled = scaler.transform(X_test)

     print("Before scaling:")
     print(X_train.iloc[:5, :5])

     print("\nAfter scaling:")
     print(X_train_scaled[:5, :5])
```

Before scaling:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness
546	10.32	16.35	65.31	324.9	0.09434
432	20.18	19.54	133.80	1250.0	0.11330
174	10.66	15.15	67.49	349.6	0.08792
221	13.56	13.90	88.59	561.3	0.10510

289 11.37 18.89 72.17 396.0 0.08713

After scaling:

```
[[-1.07200079 -0.6584246 -1.0880801 -0.93927364 -0.13593988]
 [ 1.74874285  0.06650173  1.75115682  1.74555856  1.27446827]
 [-0.97473376 -0.93112416 -0.99770871 -0.86758911 -0.61351479]
 [-0.14510328 -1.2151862 -0.12301315 -0.25319178  0.66448162]
 [-0.77161734 -0.08121053 -0.8037004 -0.73292668 -0.6722818 ]]
```

```
[7]: from sklearn.linear_model import LogisticRegression
```

```
log_model = LogisticRegression(max_iter=1000)
```

```
log_model.fit(X_train_scaled, y_train)
```

```
y_pred_log = log_model.predict(X_test_scaled)
```

```
print("Logistic Regression predictions:")
```

```
print(y_pred_log)
```

Logistic Regression predictions:

```
[0 1 0 1 0 1 1 0 0 0 1 0 1 0 0 1 0 1 1 1 0 0 1 1 1 1 0 1 1 1 1 1 1 0 0 1
 1 1 0 1 1 1 0 0 1 1 1 1 0 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 1 1 1 1 1 0
 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 1 1 1 0 0 0 1 0 1 0 1 0 0 0 1 0 0 1 0 1 0 1
 0 1 1]
```

```
[8]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
import matplotlib.pyplot as plt
```

```
accuracy_log = accuracy_score(y_test, y_pred_log)
```

```
precision_log = precision_score(y_test, y_pred_log)
```

```
recall_log = recall_score(y_test, y_pred_log)
```

```
f1_log = f1_score(y_test, y_pred_log)
```

```
print("Logistic Regression Results")
```

```
print("Accuracy :", accuracy_log)
```

```
print("Precision:", precision_log)
```

```
print("Recall   :", recall_log)
```

```
print("F1-Score :", f1_log)
```

```
cm_log = confusion_matrix(y_test, y_pred_log)
```

```
print("\nConfusion Matrix:")
```

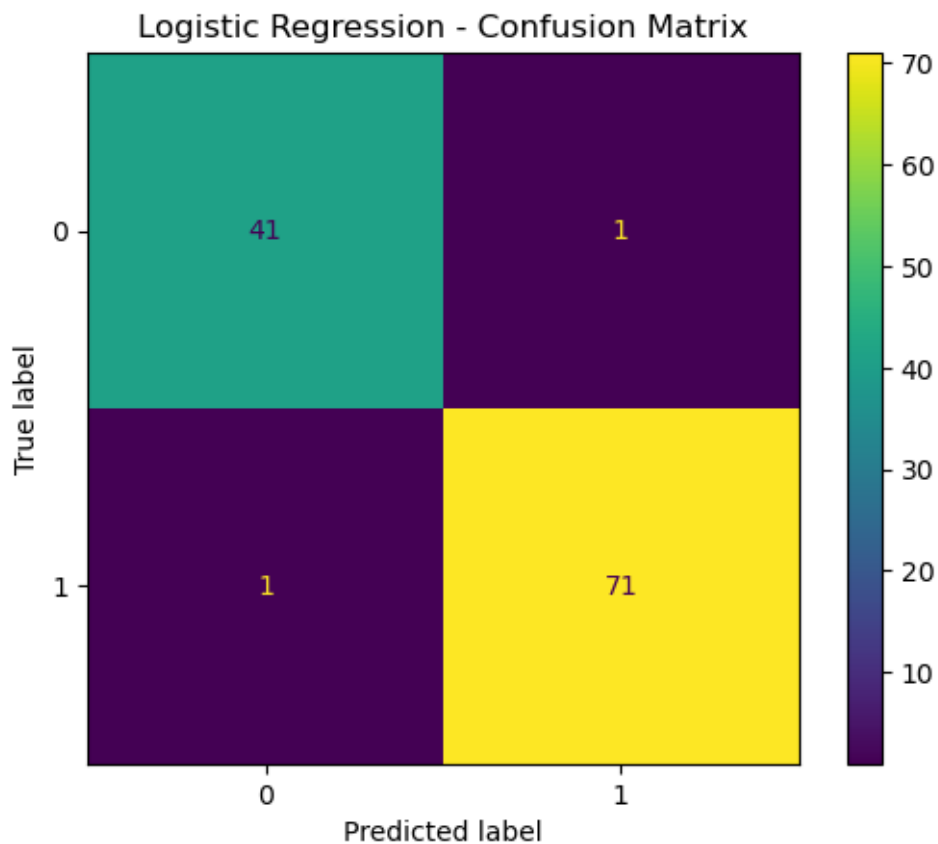
```
print(cm_log)
```

```
ConfusionMatrixDisplay(confusion_matrix=cm_log).plot()
```

```
plt.title("Logistic Regression - Confusion Matrix")
plt.show()
```

Logistic Regression Results
Accuracy : 0.9824561403508771
Precision: 0.9861111111111112
Recall : 0.9861111111111112
F1-Score : 0.9861111111111112

Confusion Matrix:
[[41 1]
 [1 71]]



```
[9]: from sklearn.neighbors import KNeighborsClassifier

knn_model = KNeighborsClassifier(n_neighbors=5)

knn_model.fit(X_train_scaled, y_train)

y_pred_knn = knn_model.predict(X_test_scaled)
```

```
print("KNN (K=5) predictions:")
print(y_pred_knn)
```

KNN (K=5) predictions:

```
[0 1 0 0 0 1 1 0 0 0 1 0 1 0 0 1 1 1 1 1 0 0 1 1 1 1 0 1 1 1 1 1 1 0 1 1
 1 0 0 1 1 1 0 0 1 1 1 1 0 1 1 1 1 1 1 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 1 0
 0 0 0 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 0 0 1 0 1 0 1 0 0 0 1 0 0 1 0 1 0 1
 0 1 1]
```

```
[10]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
      from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
      import matplotlib.pyplot as plt

      accuracy_knn = accuracy_score(y_test, y_pred_knn)
      precision_knn = precision_score(y_test, y_pred_knn)
      recall_knn = recall_score(y_test, y_pred_knn)
      f1_knn = f1_score(y_test, y_pred_knn)

      print("KNN (K=5) Results")
      print("Accuracy :", accuracy_knn)
      print("Precision:", precision_knn)
      print("Recall   :", recall_knn)
      print("F1-Score :", f1_knn)

      cm_knn = confusion_matrix(y_test, y_pred_knn)

      print("\nConfusion Matrix:")
      print(cm_knn)

      ConfusionMatrixDisplay(confusion_matrix=cm_knn).plot()
      plt.title("KNN (K=5) - Confusion Matrix")
      plt.show()
```

KNN (K=5) Results

Accuracy : 0.956140350877193

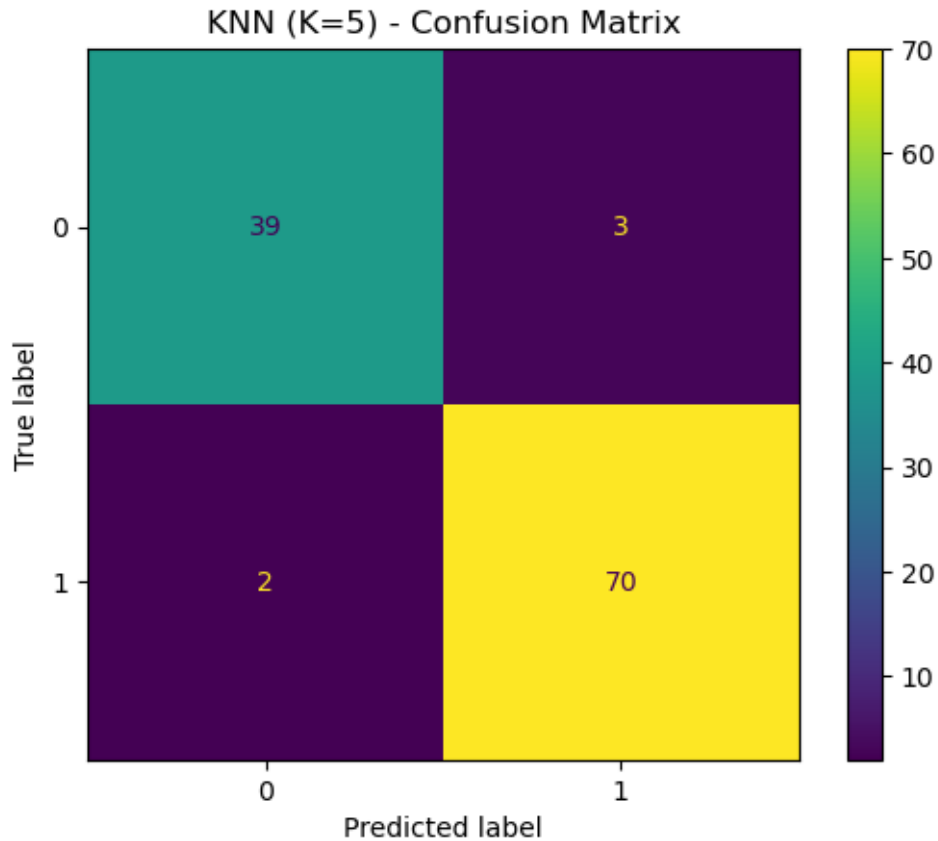
Precision: 0.958904109589041

Recall : 0.9722222222222222

F1-Score : 0.9655172413793104

Confusion Matrix:

```
[[39  3]
 [ 2 70]]
```



```
[11]: results = pd.DataFrame({
    "Model": ["Logistic Regression", "KNN (K=5)"],
    "Accuracy": [accuracy_log, accuracy_knn],
    "Precision": [precision_log, precision_knn],
    "Recall": [recall_log, recall_knn],
    "F1-Score": [f1_log, f1_knn]
})

print("Model Performance Comparison")
print(results.round(4).to_string(index=False))
```

```
Model Performance Comparison
      Model  Accuracy  Precision  Recall  F1-Score
Logistic Regression    0.9825    0.9861  0.9861    0.9861
      KNN (K=5)    0.9561    0.9589  0.9722    0.9655
```

```
[12]: import matplotlib.pyplot as plt

metrics = ["Accuracy", "Precision", "Recall", "F1-Score"]
```

```

log_values = [
    accuracy_log,
    precision_log,
    recall_log,
    f1_log
]

knn_values = [
    accuracy_knn,
    precision_knn,
    recall_knn,
    f1_knn
]

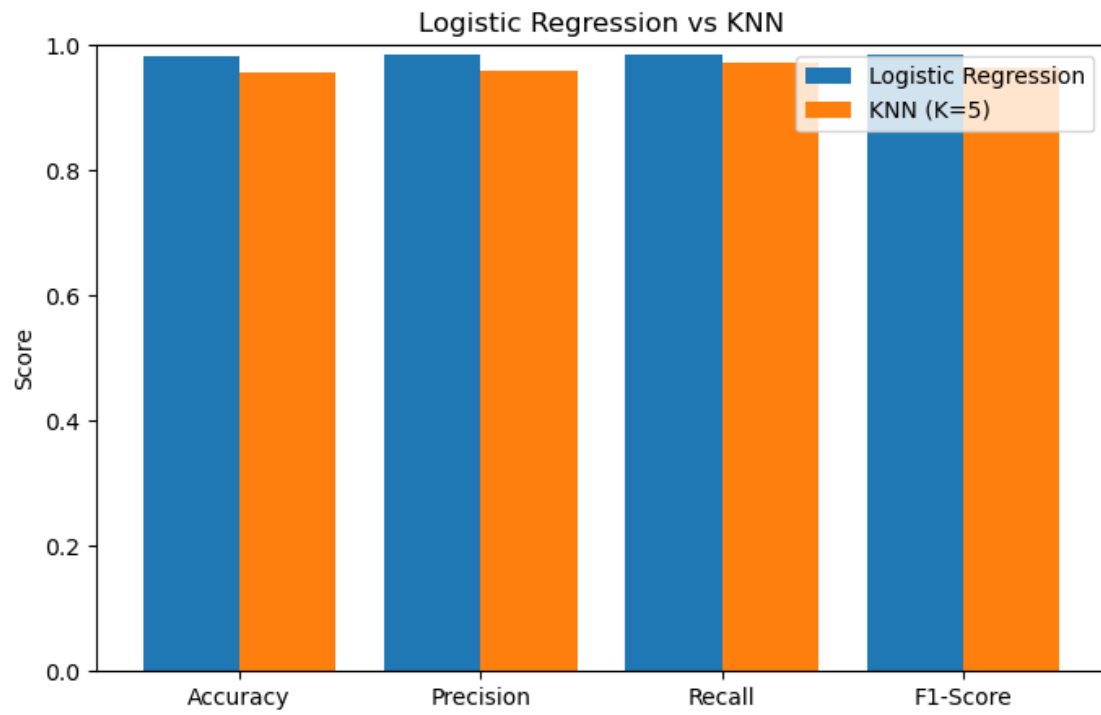
x = range(len(metrics))

plt.figure(figsize=(8, 5))

plt.bar([i - 0.2 for i in x], log_values, width=0.4, label="Logistic_Regression")
plt.bar([i + 0.2 for i in x], knn_values, width=0.4, label="KNN (K=5)")

plt.xticks(x, metrics)
plt.ylabel("Score")
plt.title("Logistic Regression vs KNN")
plt.legend()
plt.ylim(0, 1)
plt.show()

```



[]: